

tokenizer

```

"""
RegexTokenizer đơn giản (giữ chữ & số, hỗ trợ Unicode).
"""
import re
from typing import List

class RegexTokenizer:
    def __init__(self, lowercase: bool = True):
        self.lowercase = lowercase
        self._pattern = re.compile(r"[A-Za-zÀ-ÖØ-öø-ÿ0-9]+", re.UNICODE)

    def tokenize(self, text: str) -> List[str]:
        if text is None:
            return []
        if self.lowercase:
            text = text.lower()
        return self._pattern.findall(text)

```

classifier

```

"""
TextClassifier dùng Logistic Regression cho phân loại văn bản.
Ngôn ngữ chú thích: TIẾNG VIỆT (theo yêu cầu nộp bài).

API theo rubric Nhiệm vụ 1:
- fit(texts, labels): huấn luyện LogisticRegression trên đặc trưng do vectorizer tạo ra
- predict(texts): dự đoán nhãn 0/1
- evaluate(y_true, y_pred): trả về accuracy/precision/recall/f1

Bổ sung tiện ích:
- predict_proba(texts): trả về xác suất lớp 1
- save(path) / load(path): lưu/tải mô hình và vectorizer (joblib)

Mô-đun này không phụ thuộc vào cách tạo đặc trưng (bag-of-words, tf-idf,...).
Bạn truyền một "vectorizer" có 2 hàm: fit_transform(list[str]) và transform(list[str]).
"""
from __future__ import annotations
from dataclasses import dataclass
from typing import Any, List, Optional, Dict

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import joblib

@dataclass
class LRConfig:
    """Cấu hình hyper-parameters của Logistic Regression (dễ báo cáo)."""
    solver: str = "liblinear" # tốt cho tập dữ liệu nhỏ
    C: float = 1.0 # nghịch đảo độ mạnh regularization
    penalty: str = "l2"
    max_iter: int = 1000
    class_weight: Optional[str] = None # ví dụ "balanced" nếu lệch nhãn

class TextClassifier:
    """
    Trình bao (wrapper) quanh LogisticRegression của scikit-learn, tương thích vectorizer tùy chọn.
    """
    def __init__(self, vectorizer: Any, config: LRConfig = LRConfig(), random_state: Optional[int] = 42):
        self.vectorizer = vectorizer
        self.config = config
        self.random_state = random_state
        self._model: Optional[LogisticRegression] = None
        self._is_fitted: bool = False

    # ----- API chính (đúng rubric) -----
    def fit(self, texts: List[str], labels: List[int]) -> "TextClassifier":

```

```

    if not isinstance(texts, list) or not isinstance(labels, list):
        raise TypeError("texts và labels phải là list")
    if len(texts) != len(labels):
        raise ValueError("texts và labels phải có cùng độ dài")
    if len(texts) == 0:
        raise ValueError("không có dữ liệu huấn luyện")

    X = self.vectorizer.fit_transform(texts)
    y = np.asarray(labels, dtype=int)

    self._model = LogisticRegression(
        solver=self.config.solver,
        C=self.config.C,
        penalty=self.config.penalty,
        max_iter=self.config.max_iter,
        class_weight=self.config.class_weight,
        random_state=self.random_state,
    )
    self._model.fit(X, y)
    self._is_fitted = True
    return self

def predict(self, texts: List[str]) -> List[int]:
    self._ensure_fitted()
    X = self.vectorizer.transform(texts)
    return self._model.predict(X).astype(int).tolist()

def evaluate(self, y_true: List[int], y_pred: List[int]) -> Dict[str, float]:
    if len(y_true) != len(y_pred):
        raise ValueError("y_true và y_pred có độ dài khác nhau")
    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
    }

# ----- Tiện ích -----
def predict_proba(self, texts: List[str]) -> List[float]:
    self._ensure_fitted()
    X = self.vectorizer.transform(texts)
    return self._model.predict_proba(X[:, 1]).astype(float).tolist()

def save(self, path: str) -> None:
    self._ensure_fitted()
    payload = {
        "model": self._model,
        "vectorizer": self.vectorizer,
        "config": self.config,
        "random_state": self.random_state,
    }
    joblib.dump(payload, path)

@staticmethod
def load(path: str) -> "TextClassifier":
    payload = joblib.load(path)
    clf = TextClassifier(
        vectorizer=payload["vectorizer"],
        config=payload["config"],
        random_state=payload["random_state"],
    )
    clf._model = payload["model"]
    clf._is_fitted = True
    return clf

# ----- Nội bộ -----
def _ensure_fitted(self) -> None:
    if not self._is_fitted or self._model is None:
        raise RuntimeError("Mô hình chưa được huấn luyện. Hãy gọi fit(...) trước.")

```

vectorizer

```

"""
Wrapper mỏng quanh TfidfVectorizer của sklearn để cấm tokenizer tùy chọn.

```

```

"""
from typing import List, Iterable, Optional, Callable, Any
from sklearn.feature_extraction.text import TfidfVectorizer as SkTfidfVectorizer

class TfidfVectorizer:
    def __init__(
        self,
        tokenizer: Optional[Any] = None,
        ngram_range=(1,1),
        min_df: int = 1,
        max_df: float = 1.0,
        lowercase: bool = True,
        stop_words: Optional[Iterable[str]] = None,
    ):
        self._external_tokenizer = tokenizer
        analyzer: Optional[Callable[[str], List[str]]] = None
        if tokenizer is not None:
            analyzer = tokenizer.tokenize

        self._vec = SkTfidfVectorizer(
            analyzer=analyzer,
            ngram_range=ngram_range,
            min_df=min_df,
            max_df=max_df,
            lowercase=lowercase if analyzer is None else False,
            stop_words=stop_words,
        )

    def fit_transform(self, texts: List[str]):
        return self._vec.fit_transform(texts)

    def transform(self, texts: List[str]):
        return self._vec.transform(texts)

```

lab5_test

```

# -*- coding: utf-8 -*-
"""
BASELINE (NO src/): LogisticRegression + TFIDF (1-gram)
- Tự load data từ data/imdb_50k.csv nếu có; else dùng demo_data.
- In: accuracy, F1, ROC-AUC (nếu nhị phân), classification report,
      confusion matrix, 10 mẫu dự đoán, top n-grams (feature importance), 5-fold CV.
"""

import os, csv, random, re
import numpy as np
from typing import Tuple, List
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (accuracy_score, f1_score, roc_auc_score,
                             classification_report, confusion_matrix)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

random.seed(42)
np.random.seed(42)

# ----- Data -----
def _exists_large_csv():
    base = os.getcwd()
    path = os.path.abspath(os.path.join(base, "data", "imdb_50k.csv"))
    return path if os.path.isfile(path) else None

def _read_csv_guess_cols(csv_path: str, limit: int = 6000):
    """Đọc CSV có thể có cột (text,label) hoặc (review,sentiment)."""
    texts, labels = [], []
    with open(csv_path, encoding="utf-8") as f:
        reader = csv.DictReader(f)
        cols = [c.lower() for c in reader.fieldnames]
        tcol = "text" if "text" in cols else "review"
        lcol = "label" if "label" in cols else "sentiment"
        for i, row in enumerate(reader):
            if limit and i >= limit: break
            txt = row.get(tcol) or row.get(tcol.title()) or row.get("text")
            lab = row.get(lcol) or row.get(lcol.title()) or row.get("label")

```

```

        if txt is None or lab is None:
            continue
        try:
            y = int(lab)
        except:
            y = 1 if str(lab).strip().lower() in {"pos","positive","1","true"} else 0
        texts.append(txt)
        labels.append(y)
    return texts, np.array(labels, dtype=int)

def _demo_data() -> Tuple[List[str], np.ndarray]:
    pos = [
        "I absolutely loved this movie, it was fantastic!",
        "What a wonderful experience, highly recommend.",
        "Great performances and beautiful soundtrack.",
        "The plot was tight and the ending was satisfying.",
        "A charming cast and witty dialog kept me smiling.",
        "Surprisingly emotional and very well directed.",
        "The visuals are stunning and the pacing is great.",
        "Heartwarming story with believable characters.",
        "Solid film with strong acting and clever twists.",
        "Brilliant! I would watch it again.",
    ]
    neg = [
        "Terrible plot and bad acting. Waste of time.",
        "The film was boring and way too long.",
        "I wouldn't watch it again; it's awful.",
        "Predictable story and unfunny jokes.",
        "Messy editing and confusing scenes.",
        "Weak script and painfully slow pacing.",
        "Cheap effects and wooden performances.",
        "Cringe dialogue; I almost fell asleep.",
        "Disappointing and poorly executed.",
        "One of the worst movies I've seen.",
    ]
    # mở rộng ~60 câu (đủ để biểu diễn)
    aug = [(s,1) for s in pos] + [(s,0) for s in neg]
    extra = []
    for (s,y) in aug:
        s2 = re.sub(r'[!]', '', s.lower())
        extra.append((s2, y))
    dataset = aug + extra + [(s+"!",y) for (s,y) in aug]
    random.shuffle(dataset)
    X = [t for t,_ in dataset]
    y = np.array([lab for _,lab in dataset], dtype=int)
    return X, y

def load_data():
    csv_path = _exists_large_csv()
    if csv_path:
        X, y = _read_csv_guess_cols(csv_path, limit=6000)
        print(f"== Using CSV: {csv_path} | n={len(X)}")
    else:
        X, y = _demo_data()
        print(f"== Using built-in demo dataset | n={len(X)}")
    return X, y

# ----- Train / Eval -----
def evaluate(name, y_true, y_pred, proba1=None):
    print(f"\n=== {name} ===")
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average="binary")
    out = f"Accuracy: {acc:.4f} | F1: {f1:.4f}"
    if proba1 is not None and len(np.unique(y_true)) == 2:
        try:
            auc = roc_auc_score(y_true, proba1)
            out += f" | ROC-AUC: {auc:.4f}"
        except Exception:
            pass
    print(out)
    print("\nClassification report:\n", classification_report(y_true, y_pred, digits=4))
    print("Confusion matrix:\n", confusion_matrix(y_true, y_pred))

def top_features_tfidf(pipe: Pipeline, k=15):
    vec: TfidfVectorizer = pipe.named_steps["tfidf"]
    clf: LogisticRegression = pipe.named_steps["clf"]
    if not hasattr(clf, "coef_"):

```

```

        return
    feats = np.array(vec.get_feature_names_out())
    coefs = clf.coef_[0]
    top_pos = feats[np.argsort(coefs)[-k:][::-1]]
    top_neg = feats[np.argsort(coefs)[:k]]
    print("\nTop + coefficients:", " ".join(top_pos))
    print("Top - coefficients:", " ".join(top_neg))

def main():
    X, y = load_data()
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1,1), max_features=20000, stop_words='english')),
        ("clf", LogisticRegression(max_iter=1000)) # solver mặc định 'lbfgs' có predict_proba
    ])
    pipe.fit(Xtr, ytr)

    proba1 = pipe.predict_proba(Xte)[:,:1] if hasattr(pipe.named_steps["clf"], "predict_proba") else None
    yhat = pipe.predict(Xte)

    evaluate("Baseline (LogReg + TFIDF 1-gram)", yte, yhat, proba1)
    top_features_tfidf(pipe, 15)

    # In 10 mẫu dự đoán
    print("\nSample predictions:")
    for i in range(min(10, len(Xte))):
        p = proba1[i] if proba1 is not None else None
        pstr = f"{p:.4f}" if p is not None else "NA"
        print(f" y_true={yte[i]} y_pred={yhat[i]} prob1={pstr} | {Xte[i][:80]}")

    # K-Fold nhanh
    print("\n=== 5-Fold quick CV ===")
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    f1s = []
    for k, (tr, te) in enumerate(skf.split(X, y), 1):
        pipe.fit([X[i] for i in tr], y[tr])
        yk = pipe.predict([X[i] for i in te])
        f1s.append(f1_score(y[te], yk))
        print(f"Fold {k}: F1={f1s[-1]:.4f}")
    print(f"F1 mean={np.mean(f1s):.4f} ± {np.std(f1s):.4f}")

if __name__ == "__main__":
    main()

```

== Using built-in demo dataset | n=60

=== Baseline (LogReg + TFIDF 1-gram) ===
 Accuracy: 1.0000 | F1: 1.0000 | ROC-AUC: 1.0000

Classification report:

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	6
1	1.0000	1.0000	1.0000	6
accuracy			1.0000	12
macro avg	1.0000	1.0000	1.0000	12
weighted avg	1.0000	1.0000	1.0000	12

Confusion matrix:
[[6 0]
[0 6]]

Top + coefficients: great, brilliant, satisfying, tight, ending, stunning, visuals, loved, movie, fantastic, absolutely, watch,
 Top - coefficients: executed, disappointing, poorly, cheap, effects, wooden, long, boring, way, fell, editing, asleep, messy, sc

Sample predictions:

y_true=0	y_pred=0	prob1=0.3696		terrible plot and bad acting waste of time
y_true=1	y_pred=1	prob1=0.6481		Heartwarming story with believable characters.
y_true=1	y_pred=1	prob1=0.6617		surprisingly emotional and very well directed
y_true=0	y_pred=0	prob1=0.4280		predictable story and unfunny jokes
y_true=1	y_pred=1	prob1=0.5484		solid film with strong acting and clever twists
y_true=0	y_pred=0	prob1=0.4611		I wouldn't watch it again; it's awful.!
y_true=1	y_pred=1	prob1=0.6615		great performances and beautiful soundtrack
y_true=0	y_pred=0	prob1=0.3365		One of the worst movies I've seen.
y_true=1	y_pred=1	prob1=0.5484		Solid film with strong acting and clever twists.
y_true=0	y_pred=0	prob1=0.4611		i wouldn't watch it again; it's awful

```

=== 5-Fold quick CV ===
Fold 1: F1=1.0000
Fold 2: F1=1.0000
Fold 3: F1=1.0000
Fold 4: F1=1.0000
Fold 5: F1=1.0000
F1 mean=1.0000 ± 0.0000

```

lab5_spark_sentiment_analysis

```

# -*- coding: utf-8 -*-
"""
Spark Sentiment (NO findspark, dataset 6 câu in-memory)
Colab: pip install pyspark
Windows local: cần Java (và có thể cần winutils)
"""

import os, sys
os.environ.setdefault("PYSPARK_PYTHON", sys.executable)
os.environ.setdefault("PYSPARK_DRIVER_PYTHON", sys.executable)

from pyspark.sql import SparkSession
from pyspark.ml.feature import Tokenizer, StopWordsRemover, HashingTF, IDF
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

def load_data(spark):
    data = [
        ("This movie is fantastic and I love it!", 1),
        ("I hate this film, it's terrible.", 0),
        ("The acting was superb, a truly great experience.", 1),
        ("What a waste of time, absolutely boring.", 0),
        ("Highly recommend this, a masterpiece.", 1),
        ("Could not finish watching, so bad.", 0),
    ]
    return spark.createDataFrame(data, ["text", "label"])

def main():
    spark = SparkSession.builder.master("local[*]").appName("Spark Sentiment").getOrCreate()
    df = load_data(spark)
    train, test = df.randomSplit([0.7, 0.3], seed=42)

    tokenizer = Tokenizer(inputCol="text", outputCol="words")
    remover = StopWordsRemover(inputCol="words", outputCol="filtered")
    hashingTF = HashingTF(inputCol="filtered", outputCol="rawFeatures")
    idf = IDF(inputCol="rawFeatures", outputCol="features")
    lr = LogisticRegression(featuresCol="features", labelCol="label")

    pipeline = Pipeline(stages=[tokenizer, remover, hashingTF, idf, lr])
    model = pipeline.fit(train)
    preds = model.transform(test)

    acc = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy").evaluate(preds)
    print("\n=== Spark Sentiment Result ===")
    print("Accuracy:", acc)

    spark.stop()

if __name__ == "__main__":
    main()

```

```

=== Spark Sentiment Result ===
Accuracy: 0.5

```

lab5_improvement_test

```

# -*- coding: utf-8 -*-
"""
Improvement: TF-IDF (1-2gram) + MultinomialNB + 3-Fold
Dataset 6 câu in-memory
"""

```

```

Dataset: 6 câu (in-memory)
"""
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

texts = [
    "This movie is fantastic and I love it!",
    "I hate this film, it's terrible.",
    "The acting was superb, a truly great experience.",
    "What a waste of time, absolutely boring.",
    "Highly recommend this, a masterpiece.",
    "Could not finish watching, so bad.",
]
labels = np.array([1,0,1,0,1,0], int)

def main():
    Xtr,Xte,ytr,yte = train_test_split(texts, labels, test_size=0.33, stratify=labels, random_state=42)
    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1,2), max_features=4000, stop_words="english")),
        ("clf", MultinomialNB(alpha=0.5))
    ])
    pipe.fit(Xtr, ytr)
    yhat = pipe.predict(Xte)
    acc = accuracy_score(yte, yhat); f1 = f1_score(yte, yhat, zero_division=0)
    print("\n=== TFIDF 1-2 + MultinomialNB ===")
    print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
    print("\nClassification report:\n", classification_report(yte, yhat, digits=4, zero_division=0))
    print("Confusion matrix:\n", confusion_matrix(yte, yhat))

    print("\n=== 3-Fold CV ===")
    sk = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
    f1s=[]
    for k,(tr,te) in enumerate(sk.split(texts, labels),1):
        pipe.fit([texts[i] for i in tr], labels[tr])
        yk = pipe.predict([texts[i] for i in te])
        f1s.append(f1_score(labels[te], yk, zero_division=0))
        print(f"Fold {k}: F1={f1s[-1]:.4f}")
    print(f"F1 mean={np.mean(f1s):.4f} ± {np.std(f1s):.4f}")

if __name__ == "__main__":
    main()

```

```

=== TFIDF 1-2 + MultinomialNB ===
Accuracy: 0.5000 | F1: 0.0000

Classification report:
              precision    recall  f1-score   support

     0       0.5000      1.0000     0.6667         1
     1       0.0000      0.0000     0.0000         1

   accuracy          0.2500
  macro avg       0.2500      0.5000     0.3333         2
 weighted avg       0.2500      0.5000     0.3333         2

Confusion matrix:
[[1 0]
 [1 0]]

=== 3-Fold CV ===
Fold 1: F1=0.0000
Fold 2: F1=0.0000
Fold 3: F1=0.0000
F1 mean=0.0000 ± 0.0000

```

lab5_improvement_linearsvc

```

# -*- coding: utf-8 -*-
"""
Improvement: TF-IDF (1-2gram) + LinearSVC + GridSearch + KFold
Dataset: 6 câu (in-memory)
"""

```

```

import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline

texts = [
    "This movie is fantastic and I love it!",
    "I hate this film, it's terrible.",
    "The acting was superb, a truly great experience.",
    "What a waste of time, absolutely boring.",
    "Highly recommend this, a masterpiece.",
    "Could not finish watching, so bad.",
]
labels = np.array([1, 0, 1, 0, 1, 0], dtype=int)

def _choose_cv(y_train, max_cv=3):
    """Chọn số folds an toàn cho GridSearch theo dữ liệu train (>=2)."""
    counts = Counter(y_train)
    m = min(counts.values())
    return max(2, min(max_cv, m))

def main():
    Xtr, Xte, ytr, yte = train_test_split(
        texts, labels, test_size=0.33, stratify=labels, random_state=42
    )

    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=5000, stop_words="english")),
        ("clf", LinearSVC())
    ])

    grid = {
        "tfidf__max_features": [1000, 3000, 5000],
        "clf__C": [0.5, 1.0, 2.0]
    }

    safe_cv = _choose_cv(ytr, max_cv=3) # <-- cố định lỗi cv
    gs = GridSearchCV(pipe, grid, scoring="f1", cv=safe_cv, n_jobs=-1)
    gs.fit(Xtr, ytr)
    best = gs.best_estimator_
    print(f"Best params (cv={safe_cv}):", gs.best_params_)

    yhat = best.predict(Xte)
    acc = accuracy_score(yte, yhat)
    f1 = f1_score(yte, yhat, zero_division=0)
    print("\n=== LinearSVC + TFIDF 1-2 ===")
    print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
    print("\nClassification report:\n", classification_report(yte, yhat, digits=4, zero_division=0))
    print("Confusion matrix:\n", confusion_matrix(yte, yhat))

    # KFold trên toàn bộ 6 câu (mỗi lớp có 3) → dùng 3-fold OK
    print("\n=== 3-Fold CV (toàn bộ 6 câu) ===")
    sk = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
    f1s = []
    for k, (tr, te) in enumerate(sk.split(texts, labels), 1):
        best.fit([texts[i] for i in tr], labels[tr])
        yk = best.predict([texts[i] for i in te])
        f1s.append(f1_score(labels[te], yk, zero_division=0))
        print(f"Fold {k}: F1={f1s[-1]:.4f}")
    print(f"F1 mean={np.mean(f1s):.4f} ± {np.std(f1s):.4f}")

if __name__ == "__main__":
    main()

```

```
Best params (cv=2): {'clf__C': 0.5, 'tfidf__max_features': 1000}
```

```
=== LinearSVC + TFIDF 1-2 ===
Accuracy: 0.5000 | F1: 0.0000
```

```
Classification report:
      precision    recall  f1-score   support

0         0.5000      1.0000      0.6667         1
```


1	0.0000	0.0000	0.0000	1
accuracy			0.5000	2
macro avg	0.2500	0.5000	0.3333	2
weighted avg	0.2500	0.5000	0.3333	2

Confusion matrix:

```
[[1 0]
 [1 0]]
```

=== 3-Fold CV (toàn bộ 6 câu) ===

Fold 1: F1=0.0000

Fold 2: F1=0.6667

Fold 3: F1=0.0000

F1 mean=0.2222 ± 0.3143

lab5_improvement_cleaning

```
# -*- coding: utf-8 -*-
"""
Improvement: Cleaning + TF-IDF (1-2gram) + LogisticRegression + GridSearch
Dataset: 6 câu (in-memory) • CV CHỌN ĐỘNG (fix lỗi n_splits)
"""

import re, numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, classification_report, confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

texts = [
    "This movie is fantastic and I love it!",
    "I hate this film, it's terrible.",
    "The acting was superb, a truly great experience.",
    "What a waste of time, absolutely boring.",
    "Highly recommend this, a masterpiece.",
    "Could not finish watching, so bad.",
]
labels = np.array([1,0,1,0,1,0], dtype=int)

def clean_text(s: str) -> str:
    s = s.lower()
    s = re.sub(r"http\S+|www\.\S+", " ", s)
    s = re.sub(r"^[a-z\s']+", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def _safe_cv(y_train, max_cv=3):
    """Chọn số folds an toàn cho Stratified CV:  $2 \leq cv \leq \min(\text{counts})$ ."""
    c = Counter(y_train)
    m = min(c.values())
    return max(2, min(max_cv, m))

def main():
    X = [clean_text(t) for t in texts]; y = labels
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.33, stratify=y, random_state=42)

    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=1, max_features=5000, stop_words="english")),
        ("clf", LogisticRegression(max_iter=1000))
    ])
    grid = {
        "tfidf__max_features": [1000, 3000, 5000],
        "clf__C": [0.5, 1.0, 2.0]
    }

    # ---- GridSearch với CV động (fix lỗi) ----
    cv_gs = _safe_cv(ytr, max_cv=3)
    gs = GridSearchCV(pipe, grid, scoring="f1", cv=cv_gs, n_jobs=-1)
    gs.fit(Xtr, ytr)
    best: Pipeline = gs.best_estimator_
    print(f"Best params (cv={cv_gs}):", gs.best_params_)

    yhat = best.predict(Xte)
    proba = best.predict_proba(Xte)[:,-1]
    acc = accuracy_score(yte, yhat); f1 = f1_score(yte, yhat, zero_division=0)
    msg = f"Accuracy: {acc:.4f} | F1: {f1:.4f}"
    try: msg += f" | ROC-AUC: {roc_auc_score(yte, proba):.4f}"
```

```

except: pass
print("\n=== Cleaning + TFIDF 1-2 + LogReg ===")
print(msg)
print("\nClassification report:\n", classification_report(yte, yhat, digits=4, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(yte, yhat))

# ---- KFold trên toàn bộ 6 câu (mỗi lớp có 3) ⇒ cv=3 OK, nhưng vẫn chọn động cho chắc ----
print("\n=== Stratified KFold (full data) ===")
cv_full = _safe_cv(y, max_cv=3)
sk = StratifiedKFold(n_splits=cv_full, shuffle=True, random_state=42)
f1s=[]
for k,(tr,te) in enumerate(sk.split(X, y),1):
    best.fit([X[i] for i in tr], y[tr])
    yk = best.predict([X[i] for i in te])
    f1s.append(f1_score(y[te], yk, zero_division=0))
    print(f"Fold {k}: F1={f1s[-1]:.4f}")
print(f"F1 mean={np.mean(f1s):.4f} ± {np.std(f1s):.4f} (cv={cv_full})")

if __name__ == "__main__":
    main()

```

Best params (cv=2): {'clf__C': 1.0, 'tfidf__max_features': 1000}

=== Cleaning + TFIDF 1-2 + LogReg ===
 Accuracy: 0.5000 | F1: 0.6667 | ROC-AUC: 0.5000

Classification report:

	precision	recall	f1-score	support
0	0.0000	0.0000	0.0000	1
1	0.5000	1.0000	0.6667	1
accuracy			0.5000	2
macro avg	0.2500	0.5000	0.3333	2
weighted avg	0.2500	0.5000	0.3333	2

Confusion matrix:
 [[0 1]
 [0 1]]

=== Stratified KFold (full data) ===
 Fold 1: F1=0.6667
 Fold 2: F1=0.6667
 Fold 3: F1=0.6667
 F1 mean=0.6667 ± 0.0000 (cv=3)